

# LindormVector: A Distributed Vector Engine on a Cloud-Native Multi-Model NoSQL Database

Yan Wang  
Alibaba Cloud  
Hangzhou, China  
yeyan.wy@alibaba-inc.com

Jian Zhou  
Alibaba Cloud  
Hangzhou, China  
sandszhou.zj@alibaba-inc.com

Sai Huang  
Alibaba Cloud  
Hangzhou, China  
huangsai.hs@alibaba-inc.com

Chao Dou  
Alibaba Cloud  
Hangzhou, China  
douchao@alibaba-inc.com

Hanwen Tian  
Alibaba Cloud  
Hangzhou, China  
hanwen.thw@alibaba-inc.com

Zhijie Jiang  
Alibaba Cloud  
Hangzhou, China  
jiangzhijie.jzj@alibaba-inc.com

Zongning Zhang  
Alibaba Cloud  
Hangzhou, China  
zhangzongning.zzn@alibaba-inc.com

Xiaoqi Li  
University of Science  
and Technology of  
China  
Hefei, China  
lxq@mail.ustc.edu.cn

Zhencan Peng  
Rutgers University  
New Brunswick, New  
Jersey, USA  
zhencan.peng@rutgers.edu

Chunhui Shen  
Alibaba Cloud  
Hangzhou, China  
tianwu.sch@alibaba-inc.com

Wei Zhang  
Alibaba Cloud  
Hangzhou, China  
zwei@alibaba-inc.com

Feifei Li  
Alibaba Cloud  
Hangzhou, China  
lifeifei@alibaba-inc.com

Dong Deng\*  
Alibaba Cloud  
Hangzhou, China  
dong.deng@alibaba-inc.com

## Abstract

Vector databases have become a cornerstone of modern AI infrastructure, enabling semantic retrieval and retrieval-augmented generation (RAG) over massive unstructured datasets. However, existing systems face an inherent trade-off: specialized vector databases deliver low-latency in-memory search but struggle with scalability and integration with structured data, whereas general-purpose databases provide consistency and fault tolerance at the cost of query performance. In this paper, we present LindormVector, a distributed, scalable, and cost-efficient vector engine built natively on Lindorm, Alibaba Cloud's multi-model, cloud-native NoSQL database. LindormVector adopts a shared-storage architecture where all data is persisted in a distributed file system for elasticity, durability, and high availability. To minimize I/O operations, network communication, and memory footprint, LindormVector adopts an IVFPQ-style index tailored to Lindorm's storage model. A key distinction in our index design lies in the number of clusters  $n_{list}$ . We observe that the optimal number of clusters is often substantially larger than the conventional  $\sqrt{n}$  rule-of-thumb for a dataset of  $n$  vectors, especially when  $n$  is large. To guide the choice of  $n_{list}$ , we develop a practical cost model capturing the trade-off between centroid search and code scan costs. To efficiently support large  $n_{list}$  values, LindormVector accelerates K-Means clustering via approximate nearest neighbor search, organizes centroids with an HNSW graph, and introduces an optimized greedy graph traversal

for efficient centroid search. LindormVector further supports real-time index updates and efficient filtered and hybrid vector search. Extensive experiments on billion-scale datasets demonstrate that LindormVector consistently achieves high recall with millisecond-level latency and significantly outperforms both specialized and general-purpose vector databases including Milvus, Elasticsearch, and PGVector. LindormVector is already deployed in production across Alibaba Cloud, where the largest cluster consists of hundreds of ECS instances and manages more than 100 billion vectors.

## CCS Concepts

• **Information systems** → **Data management systems**; **Database query processing**; • **Theory of computation** → *Nearest neighbor algorithms*; • **Computer systems organization** → *Cloud computing*.

## Keywords

vector databases, approximate nearest neighbor search, IVFPQ, cloud-native database systems, distributed query processing

### ACM Reference Format:

Yan Wang, Jian Zhou, Sai Huang, Chao Dou, Hanwen Tian, Zhijie Jiang, Zongning Zhang, Xiaoqi Li, Zhencan Peng, Chunhui Shen, Wei Zhang, Feifei Li, and Dong Deng. 2026. LindormVector: A Distributed Vector Engine on a Cloud-Native Multi-Model NoSQL Database. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26)*, May 31–June 5, 2026, Bengaluru, India. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3788853.3803088>

## 1 Introduction

With the rapid advancement of large language models (LLMs) and generative AI, enterprises are increasingly demanding efficient semantic understanding and retrieval over massive unstructured data, including text, images, audio, and video. Traditional databases,

\*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMOD Companion '26, Bengaluru, India*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2450-3/2026/05  
<https://doi.org/10.1145/3788853.3803088>

designed for structured and schema-based workloads, struggle to process such semantically rich data efficiently. The emergence of representation learning technologies fundamentally changes this landscape: by embedding data into high-dimensional vectors and using vector distances to measure semantic similarity, they enable key capabilities such as Retrieval-Augmented Generation (RAG), multi-modal recommendation, and semantic search.

Consequently, vector search has become a fundamental building block of modern data infrastructure. Many mainstream databases and big-data systems have introduced vector search capabilities. These systems can be broadly categorized into two types: (1) specialized vector databases, which are designed from the ground up to manage and query vector data efficiently (e.g., Milvus [38, 42], Pinecone [5], Weaviate [8], and Qdrant [6]); and (2) generalized vector databases, which integrate vector search capabilities into existing data systems (e.g., SingleStore-V [14], PGVector [4, 47], Elasticsearch [3], GaussDB-Vector [37], CosmosDB-Vector [41]).

While specialized systems excel at low-latency, in-memory search, they often lack scalability and interoperability with traditional data management. In contrast, general-purpose systems inherit strong consistency guarantees, compatibility with existing data infrastructures, and high scalability, but typically suffer from limited performance due to the absence of native vector search integration.

In this paper, we present LindormVector, a distributed, scalable, and low-cost vector engine on Lindorm, Alibaba Cloud's multi-model, cloud-native NoSQL database. Lindorm provides a unified architecture that supports wide tables [30], full-text search, columnar store, and distributed computation within a single system. Its underlying storage layer, the Lindorm Distributed File System (LindormDFS), extends Apache HDFS with cloud-native capabilities for durability, elasticity, and high throughput. LindormVector builds upon these architectural foundations to deliver high-performance distributed vector search while preserving Lindorm's core strengths: low cost, fault tolerance, and horizontal scalability.

**1.1 Challenge.** Numerous index structures for approximate nearest neighbor search (ANNS), also known as vector search or semantic search, have been proposed, including quantization-based methods (e.g., Product Quantization [22], RabbitQ [19]) and graph-based methods (e.g., HNSW [29], NSG [17], Vamana/DiskANN [36]). However, directly deploying these techniques in a distributed, shared-storage system like Lindorm is non-trivial. The key challenge in designing LindormVector is to identify ANNS techniques that can integrate seamlessly with Lindorm's architecture, where data resides in a distributed file system and computation is horizontally scaled, and to compose them in a synergistic manner such that the resulting engine achieves performance comparable to, or even surpassing, specialized vector databases, while simultaneously maintaining Lindorm's core strengths in low cost, fault tolerance, and horizontal scalability.

Specifically, existing indexes typically assume in-memory or local-disk access, whereas Lindorm operates over a shared distributed storage layer. In this setting, minimizing I/O operations becomes essential for sustaining low query latency. Moreover, because Lindorm manages large-scale data across clusters of machines, network communication can easily emerge as a bottleneck.

To minimize I/O operations, network communication, and memory consumption, LindormVector employs an IVFPQ-style [25] vector index. A key distinction of our design lies in the number of clusters. We observe that the optimal number of clusters  $n_{list}$  is often far larger than the conventional rule-of-thumb that sets  $n_{list}$  to  $\sqrt{n}$  for a dataset of size  $n$ , especially when  $n$  is huge. This is because increasing the number of clusters beyond this heuristic produces finer-grained clusters, substantially reducing the number of quantized vectors that need to be scanned and thereby improving query efficiency without sacrificing recall. However, adopting large  $n_{list}$  values introduces new challenges: how to efficiently train K-Means with hundreds of thousands of centroids over high-dimensional vectors, and how to accelerate centroid search during query processing. LindormVector addresses these challenges through ANN-accelerated K-Means, an HNSW index over centroids, and an optimized greedy search for fast centroid search, as discussed later in the paper. Furthermore, LindormVector supports real-time index updates, as well as filtered and hybrid vector search.

In summary, this paper makes the following contributions.

- We present the design of LindormVector, a distributed, scalable, and low-cost vector engine built natively on Lindorm, Alibaba Cloud's multi-model, cloud-native NoSQL database.
- We propose a vector index tailored to Lindorm's shared-storage architecture, carefully minimizing I/O operations, network communication, and memory footprint while maintaining high query throughput and accuracy.
- We accelerate K-Means clustering via approximate nearest neighbor search, organize centroids with an HNSW graph, and introduce an optimized greedy traversal for low-latency centroid search.
- Extensive experiments on billion-scale datasets demonstrate that LindormVector consistently achieves high recall with millisecond-level latency and significantly outperforms both specialized and general-purpose vector databases including Milvus, Elasticsearch, PGVector, and Qdrant.

## 2 Background

### 2.1 Vector Search

Given a dataset  $\mathcal{D}$  of vectors and a query vector  $q$  in  $\mathbb{R}^d$ , the  $k$ -nearest neighbor problem seeks to find the  $k$  vectors in  $\mathcal{D}$  that are closest to  $q$  under a distance or similarity measure (e.g., Euclidean distance or cosine similarity). Due to the curse of dimensionality phenomenon [21], modern systems usually resort to approximate nearest neighbor search (ANNS), which efficiently retrieves  $k$  candidate vectors that are highly likely to include the true nearest neighbors while greatly reducing query latency. The effectiveness of ANNS is typically evaluated using  $\text{recall}@k$ , defined as the fraction of true nearest neighbors contained in the  $k$  retrieved vectors. Various indexes have been designed to support ANNS. Below, we briefly review a few representative methods.

**2.1.1 Quantization-based Methods.** Vector quantization reduces both computation and storage costs by operating on quantized vectors instead of full-precision vectors. In scalar quantization (SQ) [26], each dimension is encoded independently. A common variant, SQ8, divides each coordinate into 256 levels and stores only

the level index, reducing the storage requirement from  $4d$  bytes (for float32) to  $d$  bytes per vector. Product quantization (PQ) [22] splits each  $d$ -dimensional vector into  $m$  disjoint subspaces and applies K-Means [27] within each subspace to learn  $2^b$  centroids. Each sub-vector is then encoded by the index of its nearest centroid, resulting in  $m$  codes of  $b$  bits (a total of  $mb$  bits per vector). A full vector can be approximately reconstructed by concatenating the  $m$  corresponding centroids. During query processing, the query vector is partitioned in the same way, and distances to the  $2^b$  centroids in each subspace are precomputed. The approximate distance between the query and a database vector is then obtained by summing the per-subspace distances between the query and the centroids referenced by  $v$ 's PQ codes (i.e., the  $m$  codes). RaBitQ [19] is a binary quantization method that maps each  $d$ -dimensional vector to a vertex of a  $d$ -dimensional hypercube. It first applies a fixed random orthogonal rotation to the vector space and then encodes each rotated coordinate by its sign, producing a compact  $d$ -bit binary code. Quantization-based methods are often combined with the inverted-file (IVF) technique [22] to avoid scanning all quantized vectors. Specifically, IVF partitions all vectors into clusters using K-Means, typically creating about  $\sqrt{n}$  clusters for  $n$  vectors as a rule of thumb [16]. During query processing, only  $n$ probe clusters whose centroids are closest to the query vector are probed, where  $n$ probe is a tunable parameter that balances recall and query latency. Moreover, various techniques have been proposed to further accelerate code scanning. For example, PQFastScan [10] exploits Single Instruction Multiple Data (SIMD) operations to efficiently compute distances between the query vector and a batch of PQ codes in parallel, significantly reducing per-query latency.

**2.1.2 Graph-based Methods.** Proximity graph-based methods represent each data vector as a node in a graph, where nodes are connected based on heuristics to enable navigability [28]. At query time, a greedy search begins at a designated entry node and iteratively visits neighboring nodes whose distance to the query improves, maintaining a bounded candidate set to explore and terminating when no nearer node is found. Among the most widely-used variants is HNSW (Hierarchical Navigable Small World graph) [29], which organizes nodes across multiple layers: the bottom layer contains all vectors, while higher layers form progressively sparser graphs sampled from the layer below. A query starts at the top layer, performs a greedy descent toward the query point, then uses the closest node found as the entry into the next lower layer. This hierarchy enables rapid long-distance hops via upper layers, followed by precise refinement in the bottom layer. In HNSW, each node has at most  $M$  outgoing edges, selected via heuristics inspired by the Relative Neighborhood Graph (RNG) [39], resulting in an overall index size of  $O(Mn)$  for  $n$  vectors. Another variant, NSG (Navigable Small World Graph) [17], uses similar pruning rules but remains single-layer, and its construction typically requires an initial approximate  $k$ -nearest neighbor graph, making it more expensive to build than HNSW. Finally, DiskANN [36] adapts proximity graphs to large-scale disk-resident datasets by relaxing strict connectivity constraints and introducing disk-aware layouts, vector re-ordering, and pre-fetching to reduce random SSD I/O during traversal.

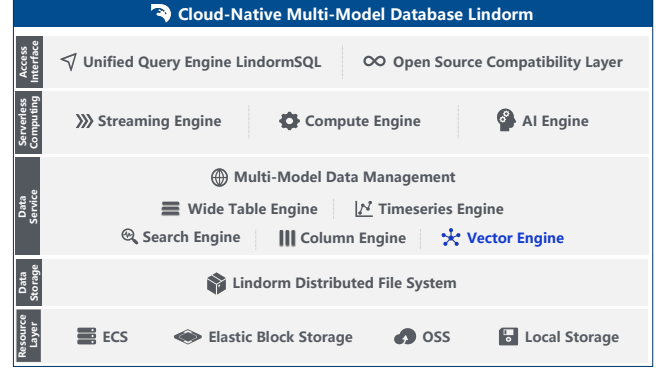


Figure 1: Overview of Lindorm.

## 2.2 Lindorm

Lindorm is a distributed, cloud-native, multi-model database designed to efficiently manage and analyze massive-scale data at low cost [30, 33]. As illustrated in Figure 1, Lindorm integrates multiple specialized engines—including wide-table, vector, search, time-series, columnar, streaming, compute, and AI engines—on top of a distributed file system. It offers a unified query interface, LindormSQL, to access data across all engines. Moreover, Lindorm is compatible with a broad range of open-source ecosystems (e.g., Apache HBase, Apache Kafka, Apache Spark, and Elasticsearch). This architecture enables integrated multi-modal data management and analytics at scale while maintaining high performance and low operational cost across diverse production workloads.

**2.2.1 Lindorm Distributed File System.** Lindorm Distributed File System (LindormDFS) is a cloud-native, scalable, and fault-tolerant storage layer shared across all Lindorm engines, providing the foundation for managing massive volumes of data. Designed to combine the strengths of both object storage and block storage, LindormDFS delivers a cost-effective, high-performance, and reliable storage service that supports elastic capacity growth. Each file in LindormDFS is partitioned into fixed-size blocks and distributed across nodes with replication to ensure durability and high availability. LindormDFS is fully compatible with the Apache HDFS protocol, allowing existing open-source ecosystems to access Lindorm data without modification. Furthermore, LindormDFS extends Apache HDFS with cloud-centric capabilities such as tiered storage for hot and cold data, intelligent data placement, and shared-replica deduplication to reduce storage overhead. By separating compute from storage, LindormDFS enables users to scale storage independently, avoiding resource coupling and lowering operational cost.

**2.2.2 Lindorm Engines.** As shown in Figure 1, Lindorm integrates many engines to enable unified querying, analyzing, and understanding of multi-modal data. The most commonly used one is the wide-table engine LindormTable. It is fully compatible with the Apache HBase API and data model, while introducing a cloud-native distributed architecture that decouples storage and compute to achieve elastic scaling from zero to petabyte scale. LindormTable organizes data by lexicographically ordered row keys. It is designed

to support high throughput point lookup and range scan by row keys over massive datasets with high scalability.

Data stored in the wide tables in LindormTable can be synchronized to other engines in real time. This architecture empowers Lindorm to support complex multi-modal queries across structured, semi-structured, and unstructured data. Specifically, the search engine provides full-text and multi-dimensional query capabilities and can serve as a secondary index for the wide tables in LindormTable, thereby accelerating retrieval and complex query workloads. The vector engine, described in this paper, adds native support for vector search. The column engine is optimized for real-time analytical queries through columnar storage, vectorized execution, and predicate pushdown, whereas the compute engine, built upon Apache Spark, is designed for large-scale batch analysis and advanced computation. Finally, the AI engine can invoke machine learning models to support in-database capabilities such as semantic embedding, text summarization, data augmentation, etc. Collectively, these engines form a unified, cloud-native platform capable of managing and analyzing multi-modal data at scale.

**2.2.3 Cluster Management and Elastic Scaling.** Lindorm adopts a shared-storage architecture that provides unified cluster management and elastic scaling for all engines. Each cluster consists of two types of nodes: LDServer and LDMaster. LDServer acts as computation nodes responsible for data management and for processing read and write requests, while the LDMaster coordinates system metadata, performs load balancing, and manages failover recovery across the cluster. Deployed in an active-standby configuration and coordinated via ZooKeeper, LDMaster ensures high availability and consistency. By horizontally scaling out LDServer, LindormVector can accommodate data at varying scales [30].

Within each engine, data is partitioned into logical units called “regions”. The regions are persisted in LindormDFS, which can be accessed by all LDServer. Partitioning is performed using a consistent hashing mechanism on the routing key, ensuring an even initial distribution of data across nodes. Each region serves as an independent unit of storage and computation that can be dynamically scheduled, migrated, and balanced across the cluster to achieve scalability and fault tolerance. LDMaster also monitors region sizes and dynamically merges or splits regions.

LDMaster also manages node scaling and failover recovery. When nodes are added or removed, LDMaster redistributes regions using consistent hashing to minimize data movement and maintain load balance. In case of node failure, LDMaster excludes the failed node and reroutes requests to healthy ones. After recovery, the node rejoins the cluster.

### 3 System Design

**3.1 Design Philosophy.** Lindorm is built to manage massive-scale enterprise data with a focus on cost efficiency, fault tolerance, and horizontal scalability. In line with this philosophy, the design goal of LindormVector is to support petabyte-scale vector data management while delivering high-performance, distributed vector search at low cost. Moreover, LindormVector cooperates with the other engines such as the search engine and the compute engine to enable hybrid and filtered vector search [31], unifying structured and unstructured data management within a single scalable system.

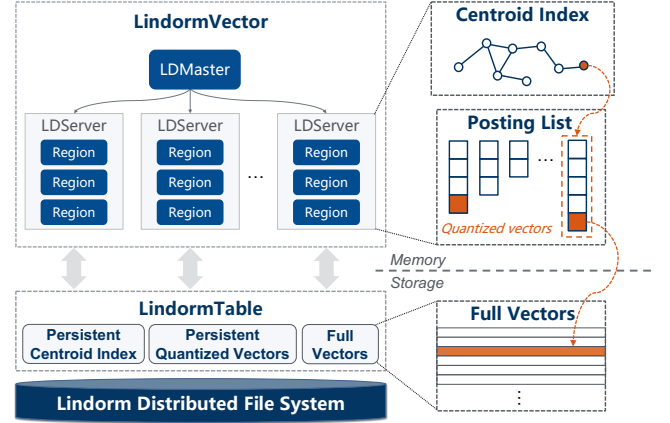


Figure 2: The architecture of LindormVector.

**3.2 Architecture.** Figure 2 shows the overall architecture of LindormVector. LindormVector is implemented as a native engine within Lindorm and is tightly integrated with LindormTable, the wide-table engine, which leverages LindormDFS to provide fault-tolerant, distributed, and elastically scalable storage. All data, including the vector index, is persisted in LindormTable. A vector index can be created on any vector column in a wide-table through a DDL (Data-Definition Language) statement, which triggers the index construction process. During this process, LindormVector scans the target column, retrieves the vectors and their corresponding row keys, and builds the index accordingly. Both LindormTable and LindormVector maintain Write-Ahead Logs (WALs) to guarantee consistency and durability across data and index updates. LindormTable partitions rows by their row keys, with each region responsible for a contiguous key range. Every region is logically managed by an LDServer and physically stored in LindormDFS. As a result, each region contains a disjoint subset of the vectors.

**3.3 Vector Index Choice.** In terms of vector index, graph-based indexes such as HNSW require all vectors to reside in memory, leading to prohibitive memory cost at scale. In line with the low-cost design goal, LindormVector focuses on disk-based vector indexing. There are two major families of disk-oriented ANNS indexes: DiskANN and IVFPQ. DiskANN repeatedly fetches full-precision vectors during graph traversal, incurring multiple rounds of get operations (i.e., I/O) from LindormTable. In contrast, IVFPQ performs nearly all computation on in-memory quantized vectors and fetches full-precision vectors only once during the final re-ranking stage, which can be efficiently supported by LindormTable’s batch get operation. The batch get operation greatly reduces network overhead compared with issuing many rounds of individual get operations. For these reasons, LindormVector adopts an IVFPQ-style index design, which minimizes I/O cost and fits naturally with Lindorm’s shared-storage architecture. Specifically, LindormVector compresses full-precision vectors using either Product Quantization (PQ) [22] or Binary Quantization (RaBitQ) [19]. They enable fast approximate distance computation while significantly reducing memory consumption. LindormVector employs SIMD (Single

Instruction, Multiple Data) to further accelerate quantized vector scanning [10–12].

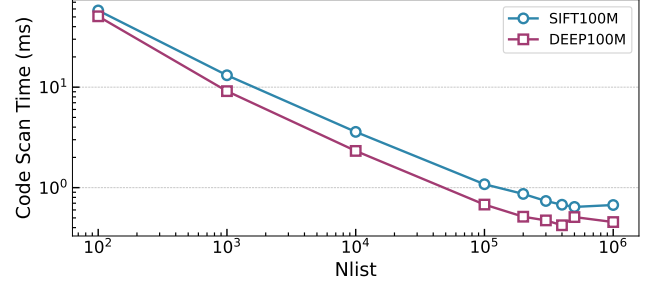
To avoid exhaustive search over all quantized vectors, LindormVector clusters the vectors using K-Means [27]. The coordinator (a process within each LDServer) maintains all cluster centroids in memory. Upon receiving a query, the coordinator identifies the  $n$ probe centroids closest to the query and broadcasts the query along with the corresponding cluster IDs to all LDServers. On each LDServer, a worker (a process within the LDServer) independently searches the quantized vectors in the selected clusters and returns a small set of top-ranked candidates to the coordinator. For efficiency, all quantized vectors are kept in memory, enabling fast distance computation without any get operation. The coordinator then aggregates the candidates returned by all LDServers, selects the most promising ones, and fetches their corresponding full-precision vectors from LindormTable using the batch get operation. Finally, it computes the exact distances between the query and these vectors and returns the top- $k$  nearest neighbors to the user.

**3.4 Vector Index Persistence.** LindormVector persists its vector index across three wide tables in LindormTable. The first table stores all centroid metadata, including an HNSW graph used for fast centroid search. The second table stores quantization metadata such as the PQ codebooks. The third table stores the quantized vectors themselves. Quantized vectors are horizontally partitioned into regions, each of which is logically managed by an LDServer and physically stored in LindormDFS with replication. During system startup, every LDServer loads the shared centroids, the quantization metadata, and the quantized vectors in its regions into memory.

A key design choice is that LindormVector employs a single, globally shared set of centroids to cluster all vectors. This ensures that quantized vectors across all regions are perfectly aligned, i.e., every region uses the same cluster boundaries. As a result, each LDServer can load the quantized vectors from all of its assigned regions and organize them into unified posting lists, where each list corresponds to a specific cluster. This global alignment greatly simplifies region management and enables seamless scaling out and scaling in across the cluster. Moreover, it mitigates hotspots caused by data or query skew, ensuring balanced memory usage and uniform search load across LDServers.

This architecture fully leverages Lindorm’s shared-storage foundation: only quantized vectors are kept in memory, while the original high-dimensional vectors remain persisted in LindormTable. As a result, LindormVector achieves an effective balance among memory consumption (i.e., cost), query latency, and query accuracy.

**3.5 Comparison with Existing Vector Databases.** The architectures of existing vector databases differ substantially from that of LindormVector. For example, Milvus [42], ElasticSearch [3], SingleStore-V [14], and Qdrant [6] organize data into segments managed by an LSM-tree-like hierarchy. Each segment contains an independent index, such as IVFPQ and HNSW. The index is persisted to local disks and loaded into memory at query time. Queries are broadcast to all segments and results are merged by a coordinator. However, this architecture suffers from several drawbacks. First, the query latency is high as every segment must be searched. Second, data freshness is low, i.e., newly inserted vectors cannot be immediately retrieved. This is because insertions either trigger



**Figure 3: Code scan time with different  $n$ list.** For each  $n$ list, we choose the smallest  $n$ probe that achieves 0.90 recall@10. We configure PQ with  $m = d$  and  $b = 4$ .

costly index construction or must wait until enough new data accumulates to build another segment. Third, periodic background segment merges cause I/O and CPU spikes, leading to unpredictable latency as new indexes are reconstructed.

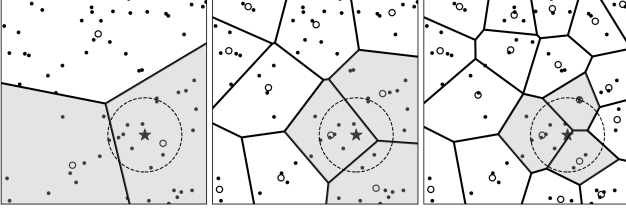
Similar to LindormVector, GaussDB-Vector [37] partitions vectors into clusters to localize search to a few clusters near the query. However, it assigns each cluster exclusively to a single node, which can lead to load imbalance: frequently queried clusters may become hotspots while others remain idle, resulting in skewed loads and increased query latency. In contrast, LindormVector randomly distributes quantized vectors across shards, which ensures balanced memory usage and uniform query load across servers.

PGVector [4] (as well as GaussDB-Vector and SingleStore-V) is built on top of traditional relational databases that provide full ACID guarantees. While these guarantees are valuable for transactional workloads, they are often unnecessary for vector data management, where performance and scalability are prioritized over strict consistency. The overhead of maintaining ACID properties—such as locking and transaction validation—significantly increases query latency, as confirmed by prior empirical studies [20]. In contrast, Lindorm adopts a design that deliberately relaxes strong transactional guarantees in favor of efficiency, scalability, and cost-effectiveness, making it a more suitable foundation for large-scale vector data management.

CosmosDB [41] adopts the DiskANN algorithm for vector search. However, DiskANN involves substantial random disk I/O operations and handles updates poorly. Harmony [45] partitions each vector into multiple sub-vectors and distributes the sub-vectors into multiple servers. This design introduces excessive network communication, which quickly becomes a scalability bottleneck. By contrast, LindormVector incurs a small amount of network and I/O costs only during the final re-ranking phase and supports real-time index updates.

## 4 Vector Index Design

A key distinction between our vector index design and the standard IVFPQ [22] lies in the number of clusters. The conventional rule of thumb sets the number of clusters to  $\sqrt{n}$  for a set of  $n$  vectors [16]. However, we observe that increasing the number of clusters beyond this heuristic produces finer-grained partitions, substantially reducing the number of quantized vectors that need to be scanned



**Figure 4: Code scan under various cluster granularity [18].**

and thereby improving query efficiency without sacrificing recall. This has also been observed in previous studies [18, 24].

We conduct an experiment to validate this observation. As shown in Figure 3, we vary  $nlist$  and measure the time spent on scanning quantized vectors (i.e., code scan) in IVFPQ. The results are reported on two widely used benchmark datasets consisting of 100 million vectors, SIFT100M [23] and DEEP100M [13]. The results show that as  $nlist$  increases, the code scan time drops quickly at first and then gradually plateaus.

Figure 4 illustrates how clustering granularity (proportional to  $nlist$ , the number of clusters) influences the efficiency of code scanning. The star denotes the query vector, and the dashed circle marks the region enclosing its topk nearest neighbors. The Voronoi cells represent clusters in the data vector space, with the shaded cell indicating the cluster that contains the topk nearest neighbors of the query. To achieve a high recall, coarse clustering (left) requires scanning a substantially larger number of quantized vectors, including many that fall outside the true nearest-neighbor region of the query. This results in increased scan time and higher query latency. As the clustering becomes finer (middle and right), the query region becomes more localized and the total number of quantized vectors scanned is considerably reduced, thereby lowering the scan cost while maintaining the same recall. However, beyond a certain point, further increasing  $nlist$  yields diminishing returns, as additional clusters exclude only a negligible number of quantized vectors. In addition, when  $nlist$  becomes excessively large, the resulting clusters become too small to benefit from SIMD-optimized batch code scanning. This can even slow down the overall code scan time. Moreover, a larger  $nlist$  substantially increases index construction cost, since the runtime of K-Means clustering grows superlinearly with the number of clusters (Section 4.2). It also increases the cost of centroid search during query processing (Section 4.3). Next, we discuss how to determine the optimal number of clusters.

#### 4.1 Identifying the Optimal Number of Clusters

To evaluate the commonly used heuristic of setting  $nlist = \sqrt{n}$  in IVFPQ, we examined its impact on the QPS–recall trade-off using SIFT100M and DEEP100M. Figure 5 reports the QPS–recall curves of our system under different values of  $nlist$ . Contrary to the heuristic, setting  $nlist = \sqrt{n} = 10K$  does not yield the best performance. Instead, increasing  $nlist$  beyond this point consistently improves the QPS–recall trade-off on both datasets, with performance continuing to rise until approximately  $nlist = 200K$ . This indicates that the optimal number of clusters is substantially larger than the classic  $\sqrt{n}$  guideline for large-scale vector search.

To identify the optimal  $nlist$ , LindormVector employs a cost model that performs a grid search over a few candidate  $nlist$  values. Specifically, we randomly sample  $m$  vectors from all the  $n$  vectors to perform K-Means clustering and select another small set (around 100) as synthetic queries. For a given  $topk$  and recall  $\rho$ , we estimate  $nprobe$ , the centroid search time, and the code scan time for each  $nlist$  as follows.

We first obtain the ground truth by comparing each synthetic query against the  $m$  sampled vectors to find its topk nearest neighbors. The  $m$  vectors are then clustered into  $nlist$  partitions using K-Means. For each query, we identify the clusters containing its ground-truth neighbors and rank their centroids by distance to the query. Let  $x$  denote the distance between the query and the  $\lceil \rho \times topk \rceil$ -th centroid. We estimate  $nprobe$  as the number of centroids whose distance to the query does not exceed  $x$ . Then we estimate the centroid search time ( $T_1$ ) and the code scan time ( $T_2$ ). For each query, we perform an actual centroid search (see Section 4.3 for more details) using the estimated  $nprobe$  and measure the execution time. The average across all queries gives the estimated  $T_1$ .

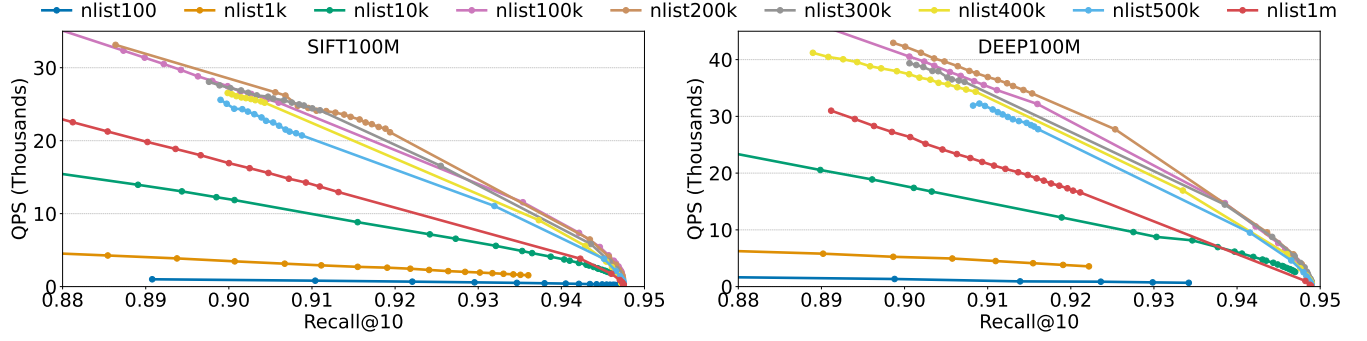
To estimate  $T_2$ , for each query, we identify the clusters whose centroids are within distance  $x$  from the query and count the total number of quantized vectors in them. This count is then scaled by the sampling ratio ( $m/n$ ) to estimate the number of codes that need to be scanned. Modern implementations process quantized vectors in fixed-size batches (e.g., 32) for SIMD efficiency. We profile the average time per batch by scanning 100K quantized vectors and measuring their processing time. Multiplying this time by the estimated number of batches yields  $T_2$ . Finally, the optimal  $nlist$  is selected as the one minimizing the total estimated search time  $T_1 + T_2$ .

**4.1.1 Empirical Evaluation.** We evaluate the effectiveness of our cost model in guiding the choice of  $nlist$  on the SIFT100M/DEEP100M datasets under various  $topk$  and recall targets. As shown in Figure 6, the estimated latency curves closely track the measured trends across all configurations. In every case, the cost model selects a near-optimal  $nlist$ , demonstrating its reliability in identifying the appropriate cluster granularity.

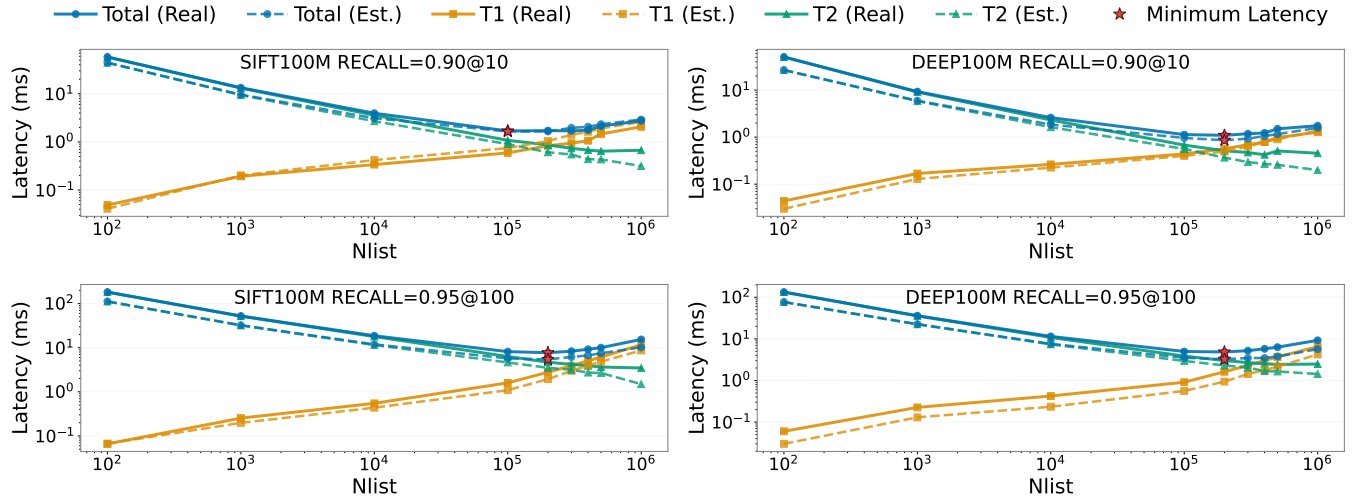
#### 4.2 Efficient Clustering with ANNS Acceleration

As discussed above, in LindormVector, the optimal number of clusters  $nlist$  typically ranges from hundreds of thousands to millions, while the vectors reside in high-dimensional spaces with dimensionality  $d$  often between 128 and 4096. Running conventional K-Means in this setting is computationally prohibitive. Each iteration of Lloyd’s algorithm requires computing distances between all training vectors and all centroids, incurring a per-iteration time complexity of  $O(md \times nlist)$  for  $m$  training vectors. This quickly becomes infeasible when both  $d$  and  $nlist$  are large, especially since the sample size  $m$  is often one to two orders of magnitude greater than  $nlist$ .

Existing K-Means optimizations are primarily designed for low-dimensional data or a moderate number of clusters. For instance, Yinyang K-Means [27] relies heavily on the triangle inequality to accelerate the clustering process. However, in high-dimensional spaces, the effectiveness of triangle inequality deteriorates rapidly



**Figure 5: QPS–recall curves for LindormVector under varying values of  $nlist$ . PQ is configured with  $m = d$  and  $b = 4$ . For both datasets, setting  $nlist$  to 200K yields the best overall performance. For high recall (above 0.94),  $nlist = 100K, 200K$ , and  $300K$  all provide similarly optimal QPS–recall trade-offs.**



**Figure 6:  $nlist$  estimation on SIFT100M/DEEP100M with two different recall@topk targets.**

due to the curse of dimensionality [21]. Consequently, these methods exhibit poor scalability when applied to large-scale clustering tasks involving hundreds of thousands of centroids over high-dimensional vectors. To address this challenge, LindormVector adopts approximate nearest neighbor search (ANNS) accelerated K-Means, which leverages efficient ANNS to accelerate the assignment step in K-Means clustering. This design substantially reduces the clustering cost while preserving the quality of the clusters.

Specifically, the algorithm starts by randomly partitioning all  $m$  training/sample vectors into  $nlist$  clusters, followed by iterative refinement until convergence. Each iteration consists of two standard steps. (1) Centroid Update: The mean vector of each cluster is computed as the new centroid. (2) ANNS-based Assignment: Instead of computing the exact distances between each vector and all centroids, we construct an in-memory HNSW index over the centroids. Each data vector is then assigned to its approximate nearest centroid through fast sublinear search. This design effectively replaces the  $O(mnlist)$  distance computations with  $O(m \log nlist)$ .

Empirically, HNSW has been shown to achieve a very high recall@1 [16]. This ensures that the clustering quality remains nearly identical to that of vanilla K-Means.

**4.2.1 Optimizations.** To further improve clustering efficiency, we adopt a few optimizations proposed in a recent study [35]. Specifically, we reuse the HNSW graph across iterations instead of rebuilding it from scratch. After the centroids are updated, each node in the graph replaces its corresponding centroid with the updated one. Because centroids typically move only slightly between iterations, the previous graph structure remains largely valid. To further refine the graph, we reinsert each centroid into the graph to update its neighbor list, using its old neighbors as entry points.

This incremental maintenance not only reduces graph reconstruction overhead but also improves graph quality, as the reinsertion step benefits from global information across all centroids. The effectiveness of multi-round HNSW construction has also been demonstrated in prior studies [46].

Furthermore, when assigning a vector to centroids using ANNS, its entry-point selection can be optimized. Since centroids change gradually between two iterations, we can use the vector's nearest centroids in the previous iteration as the entry points of the vector in the current iteration. This heuristic dramatically accelerates the convergence of HNSW greedy search.

### 4.3 Efficient Centroid Search

In this section, we discuss how to improve the performance of centroid search. Given a query, the first step in query processing is to identify the  $n$ probe centroids nearest to the query from the  $n$ list centroids. In LindormVector, the number of centroids is often extremely large. In classic IVFPQ, an exhaustive search over all centroids incurs prohibitively high latency and severely limits query throughput. To address this issue, LindormVector builds an auxiliary HNSW index over the centroids.

However, we observe that the standard HNSW index can be further improved. First of all, the main bottleneck during graph traversal in HNSW lies in the memory bandwidth, as each distance computation requires loading high-dimensional centroid vectors from memory to CPU. To reduce memory traffic while preserving accuracy, we propose replacing full-precision centroids with compressed centroids during graph traversal, followed by a lightweight reranking step using the original full-precision centroids.

Specifically, we first construct the HNSW graph using the full-precision centroids. After the graph is built, we compress each centroid in the graph by applying Principal Component Analysis (PCA) and retaining only the leading components that collectively explain 95% of the variance. During query processing, the greedy search is performed entirely on these compressed centroids. The greedy traversal returns a candidate set whose size is a multiple of  $n$ probe (namely a re-rank factor). LindormVector then retrieves the corresponding full-precision centroids, computes their exact distances to the query, and finally selects the  $n$ probe nearest ones.

Furthermore, a recent work [46] has shown that the quality of neighbor connections in a proximity graph strongly affects the performance of the resulting ANN index. Inspired by this, we introduce an additional refinement stage after the HNSW graph is built. Specifically, for each centroid, we perform an HNSW search to retrieve its nearest neighbors, apply standard pruning [29], and update the connections in the bottom layer accordingly. Experimental results show that this refinement substantially reduces the number of distance computations required to achieve a given recall target, further improving the efficiency of centroid search.

Although prior studies have reported connectivity issues in HNSW graphs [43, 48], we find that such issues are minor in our setting. Empirically, almost all centroids belong to a single strongly connected component, with only a handful of nodes forming isolated one-node components due to having zero in-degree. To address this, we simply reinsert these isolated nodes into the graph.

### 4.4 Efficient and Real-Time Vector Index Update

LindormVector supports real-time vector updates, where insertions and deletions are applied immediately to ensure data freshness. During insertion, the new vector is routed to a region by its routing key, assigned to the nearest centroid, and quantized before being

appended to the corresponding posting list. To prevent memory fragmentation, when a posting list becomes full, it is reallocated with additional pre-allocated space (16 empty slots by default); since lists are typically short, the copy overhead is minimal. For deletion, the target vector is swapped with the last entry in the posting list to maintain compactness, and the list length is decremented. An update is considered committed only after both LindormTable and LindormVector have securely persisted their WAL entries, guaranteeing consistency and durability across failures.

### 4.5 Hybrid Search and Filtered Search

**4.5.1 Hybrid Search.** Hybrid search integrates vector search and full-text search to provide results that are both semantically meaningful and lexically precise. In Lindorm, hybrid search is jointly powered by LindormVector and LindormSearch. LindormVector retrieves candidates based on vector similarity, while LindormSearch retrieves candidates using lexical matching techniques such as inverted indexes and relevance scoring (e.g., BM25). The two result sets are then fused using Reciprocal Rank Fusion (RRF) [15], a lightweight yet effective rank aggregation algorithm.

**4.5.2 Filtered Search.** Filtered search combines vector search with a predicate on structured attributes such as category, timestamp, or user ID [31, 50]. Upon receiving a filtered search query, Lindorm first estimates the selectivity of the predicate using system statistics. Based on the estimated selectivity, the optimizer dynamically chooses between pre-filtering and post-filtering. The optimizer can also be guided by user-specified hints in the query statement.

**4.5.3 Pre-filtering.** When the predicate is highly selective, causing only a small number of rows to satisfy the condition, LindormVector adopts the pre-filtering strategy. In this mode, the predicate is evaluated first using Lindorm's multi-modal engines (e.g., the search engine LindormSearch, the wide-table engine LindormTable, or the column engine). The qualified rows are retrieved, and their vectors are compared with the query vector to compute similarity scores. Finally, the topk most similar ones among the qualified rows are returned as the query results.

**4.5.4 Post-filtering.** When the selectivity of the predicate is low, causing a large number of rows to satisfy the condition, LindormVector adopts the post-filtering strategy. It performs standard vector search first and evaluates the predicate afterward at the coordinator. After aggregating candidate results returned from all workers, the coordinator fetches the corresponding rows, evaluates the predicate on these rows, and returns the topk results that satisfy the predicate. If the number of qualified results is insufficient, LindormVector increases  $n$ probe, dispatches additional centroids to the workers, and repeats the retrieval procedure until enough candidates are collected.

**4.5.5 Attribute Pushdown.** To minimize the I/O cost of fetching rows from LindormTable during query execution, LindormVector supports attribute pushdown, which embeds selected structured attributes directly into the vector index. During index construction, LindormVector stores the structured attributes of the quantized vectors in each cluster in a columnar file co-located with the posting

list of quantized vectors. At query time, when scanning a cluster, LindormVector first evaluates the predicate using the columnar file (leveraging vectorized execution for high efficiency) and then performs approximate distance computations only on the quantized vectors that satisfy the predicate. This design effectively reduces I/O overhead and accelerates filtered search performance. However, if the query predicate involves attributes that are not included in the pushed-down set, this optimization cannot be applied. In such cases, LindormVector reverts to either the pre-filtering or post-filtering strategy.

## 4.6 Ad-Hoc Optimizations

**4.6.1 Index Metadata Sharing.** LindormVector supports sharing metadata (including the centroids and the HNSW graph index over centroids) across multiple vector indexes, enabling a single set of centroids and PQ codebooks to serve several tables with identical schemas, which is especially valuable for time-partitioned or rolling tables (e.g., monthly tables). This design not only eliminates the costly K-Means and PQ training steps but also mitigates the cold-start problem, allowing newly created tables to perform accurate vector search immediately. Furthermore, by de-duplicating shared metadata in memory, LindormVector significantly lowers the memory footprint.

**4.6.2 Multi-Tenant Optimization.** A common application scenario involves platforms serving millions of users, where each user (tenant) maintains a personal collection of vectors. In practice, the majority of tenants store only a small number of vectors, while a small fraction—large tenants—possess substantial datasets. To efficiently support this highly skewed distribution, LindormVector introduces a multi-tenant vector indexing mechanism tailored for mixed tenant workloads. In LindormVector, small tenants' vectors are directly quantized and stored as individual inverted lists while sharing a global PQ codebook across all tenants. For large tenants, LindormVector dynamically promotes their data to a dedicated IVFPQ index, ensuring high recall and low latency while maintaining resource efficiency across millions of tenants.

## 5 Experiment

### 5.1 Settings

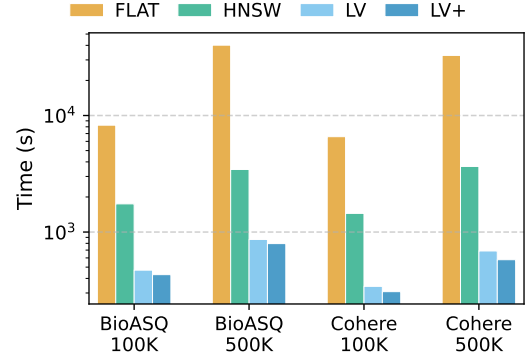
In this section, we compare LindormVector against several state-of-the-art vector databases, with an emphasis on disk-based indexes.

**5.1.1 Environment.** All experiments were conducted on Alibaba Cloud Elastic Compute Service (ECS). Each node was provisioned with an ecs.g9i.8xlarge instance (3.2 GHz Intel® Xeon® Granite Rapids processor, 32 vCPUs, 128 GB memory) with ESSD PL1 block storage (maximum IOPS 50K per disk and throughput 350MB/s per disk).

**5.1.2 Datasets.** We evaluated LindormVector using a diverse collection of publicly available datasets. (1) Cohere-10M contains 10 million embeddings generated from Wikipedia passages using Cohere's multilingual embedding model[32]. (2) BioASQ-10M consists of 10 million biomedical embeddings from the BioASQ challenge series on large-scale semantic indexing and question answering [40]. (3) Turing-1B is a 1-billion-vector dataset released under

**Table 1: Summary of Evaluation Datasets.**

| Scale       | Name         | Count  | Dim. (d) | Raw Size |
|-------------|--------------|--------|----------|----------|
| Medium      | Cohere-10M   | $10^7$ | 768      | 30GB     |
|             | BioASQ-10M   | $10^7$ | 1024     | 40GB     |
| Large       | Turing-1B    | $10^9$ | 100      | 400GB    |
|             | SpaceV-1B    | $10^9$ | 100      | 400GB    |
| Ultra-Large | Synthetic-1B | $10^9$ | 1024     | 4TB      |



**Figure 7: K-Means clustering time over 10 million vectors.**

**Table 2: Comparing the Mean Squared Errors of K-Means.**

| Datasets & nlist | BioASQ-10M 100K        | BioASQ-10M 500K        | Cohere-10M 100K     | Cohere-10M 500K     |
|------------------|------------------------|------------------------|---------------------|---------------------|
| FLAT             | $3.474 \times 10^{11}$ | $2.908 \times 10^{11}$ | $1.823 \times 10^6$ | $1.518 \times 10^6$ |
| HNSW             | $3.474 \times 10^{11}$ | $2.917 \times 10^{11}$ | $1.829 \times 10^6$ | $1.549 \times 10^6$ |
| LV               | $3.475 \times 10^{11}$ | $2.908 \times 10^{11}$ | $1.856 \times 10^6$ | $1.521 \times 10^6$ |
| LV+              | $3.474 \times 10^{11}$ | $2.906 \times 10^{11}$ | $1.855 \times 10^6$ | $1.515 \times 10^6$ |

Microsoft's Turing initiative for modeling large-scale web-search intent [34]. (4) SpaceV-1B, also released by Microsoft Bing, contains 1 billion vectors capturing user intent and behavioral patterns in commercial search engines [34]. Finally, (5) Synthetic-1B is a synthetic 1-billion-vector dataset generated by injecting Gaussian noise into BioASQ-10M, providing a controlled benchmark for evaluating scalability. For each dataset, we randomly sampled 1,000 vectors as the query set. Table 1 summarizes the key characteristics of all datasets used in our evaluation.

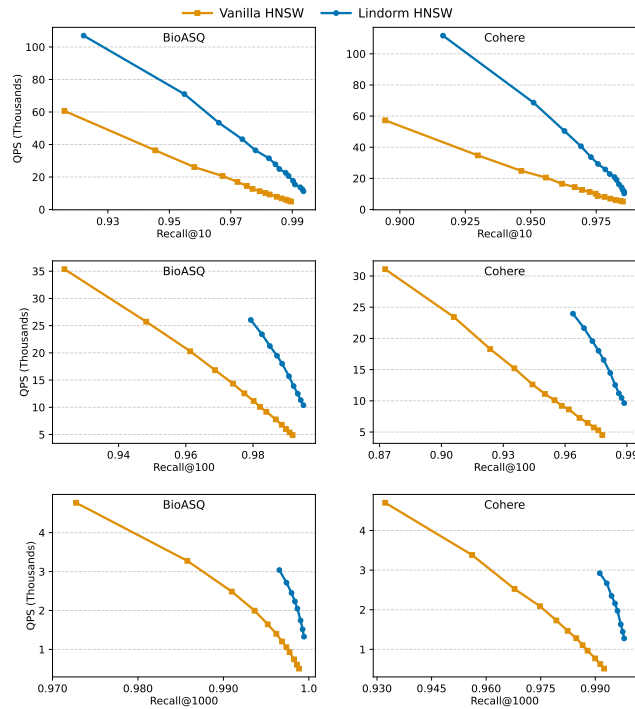
### 5.2 Evaluating Clustering and Centroid Search

**5.2.1 K-Means Clustering.** We evaluate four K-Means algorithms on the two medium-scale datasets Cohere-10M and BioASQ-10M: (1) FLAT, the classical K-Means implementation with exact (brute-force) nearest-centroid assignment; (2) HNSW, which uses the HNSW implementation in Faiss [16] for cluster assignment; (3) LV, which uses LindormVector's optimized centroid search algorithm (Section 4.3); and (4) LV+, which further incorporates the entry-point selection heuristic introduced in Section 4.2.

We used all 10 million vectors for K-Means training and evaluated two  $nlist$  values, 100,000 and 500,000. Figure 7 shows the end-to-end clustering time. As shown, HNSW yields substantial performance improvements over FLAT, especially when the number of clusters  $nlist$  is large. For example, when  $nlist = 100,000$ , HNSW reduces clustering time by 68%–78% compared with FLAT; when  $nlist = 500,000$ , the reduction increases to 91%. Building on this, LindormVector’s optimized centroid-search algorithm (LV) provides an additional 40%–81% speedup relative to HNSW, while the LV+ entry-point optimization yields a further 7%–15% improvement over LV.

Despite these substantial reductions in clustering time, the clustering quality remains effectively unchanged. Table 2 reports the mean squared error (MSE) of the resulting clusters, showing that all ANNS-accelerated methods achieve MSE values nearly identical to FLAT, with only negligible fluctuations.

Overall, ANNS-accelerated K-Means reduces clustering time by 5–10 $\times$  with negligible quality loss, enabling fine-grained clustering at billion-vector scale.



**Figure 8: Evaluating the optimized centroid search algorithm on 500K centroids.**

**5.2.2 Centroid Search.** We evaluate the performance of our optimized centroid search algorithm (Section 4.3) using the 500K centroids trained earlier for the two medium-scale datasets. We measure  $k$ -nearest-centroid search for  $k \in \{10, 100, 1000\}$ . After applying PCA that retains 95% of the variance, the centroid dimensionality is reduced to 384 for BioASQ and 320 for Cohere. We use

a rerank factor of 3. Figure 8 compares the Faiss HNSW implementation (denoted as Vanilla HNSW) with our optimized algorithm (denoted as Lindorm HNSW).

Across all  $k$  values and both datasets, our optimized algorithm consistently offers a superior QPS–recall trade-off compared to Vanilla HNSW. Specifically, under the same recall level, QPS improves by 32% to 154% for  $k = 10$ , 25% to 165% for  $k = 100$ , and 68% to 349% for  $k = 1000$ . These results indicate multi-fold centroid-search speedups at matched recall, making large-scale centroid search practical even at very fine granularities.

### 5.3 Comparison with Existing Vector Databases

**5.3.1 Baselines and Parameters.** We compared LindormVector with several representative open-source vector databases, including Milvus, Elasticsearch, PGVector, and Qdrant. We focus on workloads that cannot fit in memory and thus evaluate only the disk-based indexes of these systems.

Specifically, for Milvus, we used the latest stable release (version 2.6.4) and evaluated its DiskANN index<sup>1</sup>. The main parameters were set as follows: MaxDegree = 32, SearchListSize = 60, PQCodeBudgetGBRatio = 0.125, BuildNumThreadsRatio = 1, SearchCacheBudgetGBRatio = 0.1, LoadNumThreadRatio = 8, BeamWidthRatio = 4, and gracefulTime = 5000.

We used PGVector version 17.6, which supports two index types, HNSW and IVFFLAT<sup>2</sup>. We selected IVFFLAT because building an HNSW index on PostgreSQL is prohibitively slow: inserting each new vector into the HNSW graph triggers numerous random disk I/O operations, making construction impractical for large workloads. The main parameters for the IVFFlat index were set as follows: Pre-processing = binary\_quantize, QuantizationLength = 100 bits, and DistanceOperatorClass = bit\_hamming\_ops. For medium-scale datasets, the parameter IVFFlatLists (i.e.,  $nlist$ ) was set to 3162. For the billion-scale datasets, it was set to 10,000 which is significantly below the theoretical optimum. This is because setting  $nlist$  to 1 million consistently resulted in an Out-Of-Memory (OOM) error during index construction.

Elasticsearch (version 9.2.1) recently introduced a disk-oriented index, DiskBBQ<sup>3</sup>. We therefore adopted type: bbq\_disk as the index type for Elasticsearch, with all parameters set to their default values.

Qdrant (version 1.15.5) only offers a single dense vector index, HNSW<sup>4</sup>. Thus we configured its index with the HNSW algorithm and explicitly set on\_disk: true to ensure a fair disk-based comparison. Its parameter settings were shard=1, m=32, ef\_construct=400.

LindormVector was optimized based on our cost model: on medium-scale datasets, we set  $nlist = 50,000$  with Product Quantization (PQ) set to compress each dimension into 4-bit codes (i.e.,  $m = d$  and  $b = 4$ ). For billion-scale datasets, the number of clusters was set to  $nlist = 200,000$ , and the PQ compression was configured to compress every two dimensions into 4-bit codes (i.e.,  $m = d/2$  and  $b = 4$ ).

**5.3.2 Query Performance.** We evaluate query performance on all medium- and large-scale datasets using a single-node cluster by

<sup>1</sup><https://milvus.io/docs/diskann.md>

<sup>2</sup><https://github.com/pgvector/pgvector?tab=readme-ov-file#indexing>

<sup>3</sup><https://www.elastic.co/search-labs/blog/diskbbq-elasticsearch-introduction>

<sup>4</sup><https://qdrant.tech/documentation/concepts/indexing/#vector-index>

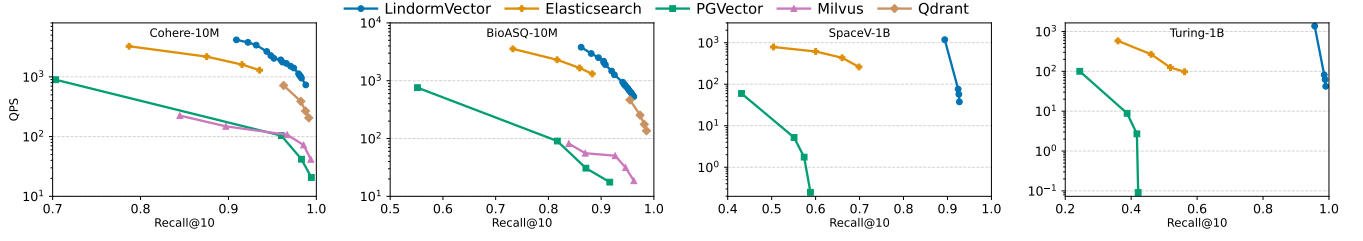


Figure 9: QPS vs. Recall@10 curves of LindormVector and the baseline systems.

Table 3: Index Time.

| System        | Cohere-10M | BioASQ-10M | Turing-1B | SpaceV-1B |
|---------------|------------|------------|-----------|-----------|
| Milvus        | 12454s     | 6270s      | 16h       | 17.5h     |
| Qdrant        | 1980s      | 2750s      | /         | /         |
| PGVector      | 1165s      | 1843s      | 3.33h     | 3.3h      |
| Elasticsearch | 4776s      | 3798s      | 19.6h     | 9.4h      |
| LindormVector | 902s       | 990s       | 16h       | 12.5h     |

varying search parameters (e.g., `nprobe` in LindormVector, `ef_search` in Qdrant, and `num_candidates` in Elasticsearch) and reporting QPS–recall curves (Figure 9). For Turing-1B and SpaceV-1B, Qdrant failed to finish index construction within 24 hours, and Milvus could not load the built index for querying due to memory limits; therefore only LindormVector, Elasticsearch, and PGVector completed end-to-end search evaluation on billion-scale data. Across all datasets, LindormVector achieves the best QPS–recall trade-off: at the same recall level, it delivers 1–2 orders of magnitude higher QPS, exceeds PGVector and Milvus by over 10× on Cohere-10M, and widens the gap further on billion-scale workloads. This gain comes from fine-grained clustering, optimized centroid search, and memory-resident quantized vectors that reduce I/O and CPU overhead. The advantage is consistent across both medium-scale and billion-scale workloads, indicating that the design remains effective as data size and system pressure grow.

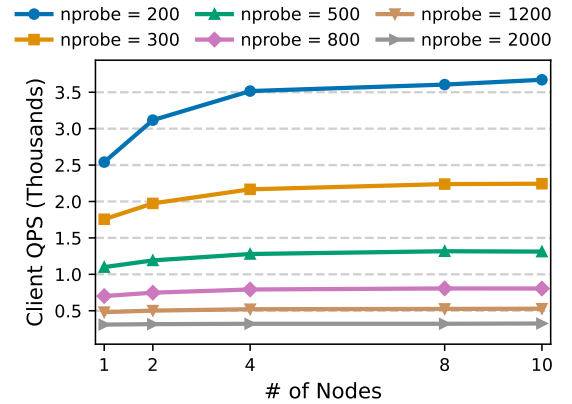
**5.3.3 Index Time.** We next evaluate index construction time (Table 3). On medium-scale datasets, LindormVector is substantially faster than existing open-source systems; for example, on BioASQ-10M it finishes in 990 seconds versus 6270 seconds for Milvus (6× faster), and on Cohere-10M it finishes in about 15 minutes while Milvus and Elasticsearch are close to one hour. For billion-scale datasets, PGVector reports the shortest build time mainly because it uses a much smaller IVFFLAT setting (`nlist` = 10,000), which lowers assignment cost at the expense of query quality. At this scale, LindormVector is slightly slower than PGVector primarily due to additional get/put overhead in the distributed storage layer.

LindormVector’s indexing efficiency mainly comes from lightweight PQ/RaBitQ with ANN-accelerated K-Means and a single global index, which avoids repeated segment-level re-indexing in LSM-style systems. This makes LindormVector well suited for production deployments that require both fast bulk loading and stable query performance after ingestion.

## 5.4 Scalability

In this section, we evaluate the scalability of LindormVector using the Synthetic-1B dataset. To measure horizontal scaling, we deploy LindormVector on clusters ranging from 1 to 10 LDServer nodes, while keeping the per-node data volume fixed at  $10^8$  vectors. Thus, the total dataset size increases proportionally from  $10^8$  to  $10^9$  vectors as the cluster grows. We use a fine-grained partitioning with `nlist` = 200,000, vary `nprobe` from 200 to 2000, fix the rerank factor at 2, and report the client-side QPS. As shown in Figure 10, LindormVector maintains remarkably stable query throughput as the cluster scales out. The QPS even increases as the cluster size grows when the `nprobe` ratio (`nprobe`/`nlist`) is small. For example, when `nprobe` = 200, the client QPS increases from roughly 2541 on the one-node cluster to 3516 on the four-node cluster and 3671 on the ten-node cluster.

This trend is mainly due to the fixed rerank factor: it bounds the number of full-precision vectors fetched from LindormTable, so the reranking work per query remains essentially constant as the cluster grows. Meanwhile, the batch-get requests for these vectors are spread across more LDServer nodes and executed in parallel, which reduces rerank latency; when `nprobe` becomes large, however, centroid search and PQ code scanning dominate the total cost, so the benefit from parallel reranking becomes less pronounced.

Figure 10: QPS as the number of nodes increases from 1 to 10, while the total dataset size grows proportionally from 100M to 1B vectors on the Synthetic-1B dataset (`nlist` = 200K).

## 5.5 Evaluating Index Update and Filtered Search

**5.5.1 Index Update.** We measured the average latency of single-vector insertion and deletion operations. On medium-scale datasets (Cohere-10M and BioASQ-10M), LindormVector achieves an average write latency of approximately 3–4 ms. On billion-scale datasets, including Turing-1B and SpaceV-1B, the update latency remains low, ranging from 33–34 ms. The overall update cost is primarily dominated by centroid search and the get/put operations issued to the distributed storage layer. Thanks to the optimized centroid-search procedure, LindormVector sustains low update overhead even at billion-vector scale. Overall, these results indicate consistently low write latency across datasets with different scales and dimensionalities, which is important for maintaining high data freshness in production workloads.

**5.5.2 Filtered Vector Search.** We evaluate filtered vector search on the Cohere-10M dataset by augmenting each vector with a scalar attribute and issuing vector queries combined with predicates of varying selectivities. We use an index with  $nlist = 50000$ . Table 4 reports the end-to-end latency (L, in milliseconds) and Recall@10 (R) for three filtering strategies: pre-filtering, post-filtering, and attribute pushdown. A dash (–) indicates that the query timed out.

Pre-filtering succeeds only at 1% selectivity and is slow due to row fetches from LindormTable; post-filtering is faster (down to 15.8 ms) but drops to 70%–75% recall for 10%–90% selectivity. In contrast, attribute pushdown is the only method that works across the full selectivity range, maintaining about 10–13 ms latency and about 95% recall. Its key advantage is that predicate evaluation is performed directly on lightweight, co-located in-memory attributes during code scanning, thereby avoiding the expensive row fetches required by pre-filtering.

**Table 4: Filtered vector search performance on Cohere-10M ( $nlist = 50000$ ). L = latency (ms), R = recall@10.**

| selectivity (%) | Pre-filter |      | Post-filter |      | Pushdown |      |
|-----------------|------------|------|-------------|------|----------|------|
|                 | L          | R    | L           | R    | L        | R    |
| 1               | 734.12     | 96.4 | —           | —    | 10.60    | 95.0 |
| 10              | —          | —    | 33.46       | 70.6 | 9.38     | 92.7 |
| 50              | —          | —    | 33.19       | 74.3 | 11.78    | 95.8 |
| 90              | —          | —    | 38.64       | 75.9 | 12.81    | 96.2 |
| 99              | —          | —    | 15.80       | 95.2 | 10.60    | 95.4 |

**5.5.3 Multi-Tenant Filtered Search.** We further evaluate a special case of filtered search where each query is restricted by a tenant identifier. On Synthetic-1B, we assign every 500 vectors to one tenant (about 2 million tenants). Under this setting, LindormVector uses only 0.35 MB index memory per tenant and achieves 3.11 ms latency with recall@10 of 0.99. These results show that LindormVector can sustain both high performance and low per-tenant memory overhead even when serving millions of tenants, making it well suited for large-scale multi-tenant vector search services.

## 6 Related Work

**6.1 Vector Databases.** There exist several specialized vector databases, such as Milvus [42], Weaviate [8], Chroma [1], Qdrant [6],

and Pinecone [5], which are designed from scratch exclusively for vector search workloads. In parallel, many general-purpose databases and search engines have introduced vector search capabilities through plugins or extensions, including Elasticsearch [3], PostgreSQL [47], ClickHouse [2], Redis [7], SingleStore [14], GaussDB-Vector [37], and CosmosDB [41]. These systems aim to integrate vector search into existing infrastructures while minimizing architectural disruption and system overhead [49].

Filtered search—i.e., vector search combined with structured predicates—is supported by all the aforementioned systems. Most systems adopt either pre-filtering (apply predicates first, then run ANNS) or post-filtering (run ANNS first, then discard non-matching items). CosmosDB takes a different approach: instead of pure post-filtering, it uses a filter-aware graph traversal that evaluates predicates into bitmaps and biases DiskANN’s search toward nodes likely to satisfy the filter [41]. GaussDB-Vector builds a dedicated hybrid index that stores scalar attributes in a balanced tree and attaches IVF or graph sub-indexes to internal nodes. While effective for predicates on that specific key, this design incurs expensive maintenance overhead—splitting or merging tree nodes may require rebuilding the corresponding vector sub-indexes [37].

**6.2 Distributed Vector Search.** As vector collections grow to billions and even trillions of embeddings, many systems provide distributed vector search. Elasticsearch [3], CosmosDB [41], and several vector database services [1, 5, 6, 8, 42] adopt a shard-per-node architecture: the collection is partitioned into independent shards, each shard builds its own ANN index, and every query is broadcast to all shards with a final merge of partial results. In contrast, ADB-V [44], Harmony [45], GaussDB-Vector [37] and our LindormVector use a cluster-based two-layer design that first searches a small set of cluster centroids and then restricts vector probing to the selected clusters; ADB-V and Harmony use a FLAT upper-layer index over centroids, whereas GaussDB-Vector employs a hierarchical tree. LindormVector instead builds one logical IVFPQ-style index whose centroids, codebooks, and centroid graph are shared across all nodes in a shared-storage architecture. DISTRIBUTEDANN [9] instead distributes a single DiskANN-style proximity graph: both the upper and lower layers are graphs, where the top layer routes queries to good entry points in the sharded base graph.

## 7 Conclusion

We presented LindormVector, a distributed, scalable, and cost-effective vector engine built natively on Lindorm, Alibaba Cloud’s multi-model, cloud-native NoSQL database. LindormVector adopts a shared-storage architecture and an IVFPQ-based index guided by a practical cost model that balances accuracy, latency, and memory efficiency. To support a very large number of clusters, LindormVector accelerates K-Means training via approximate nearest neighbor search and introduces an optimized centroid-search algorithm that delivers substantial speedups. In addition, LindormVector provides real-time index updates as well as efficient hybrid and filtered vector search. Together, these techniques allow LindormVector to achieve high recall, millisecond-level latency, and strong cost efficiency at billion-vector scale, while fitting naturally into Lindorm’s broader multi-modal data ecosystem.

## References

- [1] [n. d.]. Chroma. <https://www.trychroma.com>. Accessed: 2025-10-26.
- [2] [n. d.]. Clickhouse. <http://clickhouse.com>. Accessed: 2025-10-26.
- [3] [n. d.]. Elasticsearch. <http://elastic.co>. Accessed: 2025-10-26.
- [4] [n. d.]. PGVector. <https://github.com/pgvector/pgvector>.
- [5] [n. d.]. Pinecone. <http://pinecone.io>. Accessed: 2025-10-26.
- [6] [n. d.]. Qdrant. <https://qdrant.tech>. Accessed: 2025-10-26.
- [7] [n. d.]. Redis. <http://redis.io>. Accessed: 2025-10-26.
- [8] [n. d.]. Weaviate. <http://weaviate.io>. Accessed: 2025-10-26.
- [9] Philip Adams, Menghao Li, Shi Zhang, Li Tan, Qi Chen, Mingqin Li, Zengzhong Li, Knut Risvik, and Harsha Vardhan Simhadri. 2025. DISTRIBUTEDANN: Efficient Scaling of a Single DISKANN Graph Across Thousands of Computers. *arXiv preprint arXiv:2509.06046* (2025).
- [10] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2016. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. In *42nd International Conference on Very Large Data Bases*, Vol. 9. 12.
- [11] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2017. Accelerated nearest neighbor search with quick adc. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*. 159–166.
- [12] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2019. Quicker adc: Unlocking the hidden potential of product quantization with simd. *IEEE transactions on pattern analysis and machine intelligence* 43, 5 (2019), 1666–1677.
- [13] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.
- [14] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. 2024. Singlestore-v: An integrated vector database system in singlestore. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3772–3785.
- [15] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759.
- [16] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. *CoRR abs/2401.08281* (2024). [arXiv:2401.08281](https://arxiv.org/abs/2401.08281) doi:10.48550/ARXIV.2401.08281
- [17] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graphs. *PVLDB* 12, 5 (2019), 461 – 474. doi:10.14778/3303753.3303754
- [18] Yujian Fu, Cheng Chen, Xiaohui Chen, Weng-Fai Wong, and Bingsheng He. 2024. Optimizing the Number of Clusters for Billion-Scale Quantization-Based Nearest Neighbor Search. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 6786–6800.
- [19] Jianyang Gao and Cheng Long. 2024. Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.
- [20] Stavros Harizopoulos, Daniel J Abadi, Samuel Madden, and Michael Stonebraker. 2018. OLTP through the looking glass, and what we found there. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*. 409–439.
- [21] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.
- [22] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [23] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: re-rank with source coding. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864.
- [24] Wenqi Jiang, Shigang Li, Yu Zhu, Johannes de Fine Licht, Zhenhao He, Runbin Shi, Cedric Renggli, Shuai Zhang, Theodoros Rekatsinas, Torsten Hoefler, et al. 2023. Co-design hardware and algorithm for vector search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [25] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [26] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.
- [27] J MacQueen. 1967. Multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1. 281–297.
- [28] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [29] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *TPAMI* 42, 4 (2018), 824–836.
- [30] Qianyu Ouyang, Chunhui Shen, Wenlong Yang, Peng Yu, Qiang Xiao, Jianhui Lei, Yadong Chen, Qilu Zhong, Xiang Wang, Yong Lin, et al. 2024. Lindorm-UWC: An Ultra-Wide-Column Database for Internet of Vehicles. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4117–4129.
- [31] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024), 120. doi:10.1145/3654923
- [32] Nils Reimers and Jay Alammar. 2023. The Embedding Archives: Millions of Wikipedia Article Embeddings in Many Languages. <https://cohere.com/blog/embedding-archives-wikipedia>.
- [33] Chunhui Shen, Qianyu Ouyang, Feibo Li, Zhipeng Liu, Longcheng Zhu, Yujie Zou, Qing Su, Tianhuan Yu, Yi Yi, Jianhong Hu, et al. 2023. Lindorm TSDB: A cloud-native time-series database for large-scale monitoring systems. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3715–3727.
- [34] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, et al. 2022. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 177–189.
- [35] Jack Spalding-Jamieson, Eliot Wong Robson, and Da Wei Zheng. 2025. Scalable k-Means Clustering for Large k via Seeded Approximate Nearest-Neighbor Search. *arXiv preprint arXiv:2502.06163* (2025).
- [36] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. Curran Associates Inc., Red Hook, NY, USA.
- [37] Ji Sun, Guoliang Li, James Pan, Jiang Wang, Yongqing Xie, Ruicheng Liu, and Wen Nie. 2025. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4951–4963.
- [38] Milvus Team. 2025. Vector Search in the Real World: How to Filter Efficiently Without Killing Recall. <https://milvus.io/blog/how-to-filter-efficiently-without-killing-recall.md>. Accessed: 2025-10-17.
- [39] Godfried T Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern recognition* 12, 4 (1980), 261–268.
- [40] George Tsatsaronis, Georgios Balikas, Prodromos Malakasiotis, Ioannis Partalas, Matthias Zschunke, Michael R Alvers, Dirk Weissenborn, Anastasia Krithara, Sergios Petridis, Dimitris Polychronopoulos, et al. 2015. An overview of the BIOASQ large-scale biomedical semantic indexing and question answering competition. *BMC bioinformatics* 16, 1 (2015), 138.
- [41] Nitish Upreti, Harsha Vardhan Simhadri, Hari Sudan Sundar, Krishnan Sundaram, Samer Boshra, Balachandrar Perumalswamy, Shivam Atri, Martin Chisholm, Revti Raman Singh, Greg Yang, et al. 2025. Cost-Effective, Low Latency Vector Search with Azure Cosmos DB. *arXiv preprint arXiv:2505.05885* (2025).
- [42] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [43] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. doi:10.14778/3476249.3476255
- [44] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. Analyticdb-v: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.
- [45] Qian Xu, Feng Zhang, Chengxi Li, Lei Cao, Zheng Chen, Jidong Zhai, and Xiaoyong Du. 2025. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–28.
- [46] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2024. Revisiting the index construction of proximity graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2410.01231* (2024).
- [47] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2241–2253.
- [48] Minjia Zhang and Yuxiong He. 2018. Zoom: Ssd-based vector search for optimizing accuracy, latency and memory. *arXiv preprint arXiv:1809.04067* (2018).
- [49] Yunan Zhang, Shige Liu, and Jianguo Wang. 2024. Are there fundamental limitations in supporting vector data management in relational databases? A case study of PostgreSQL. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3640–3653.
- [50] Chaofei Zhao, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), 69:1–69:26. doi:10.1145/3639324